

OSSP Practical session-11

Task-1: Develop a multithreaded counter application using `pthread_create()` and `pthread_join()`. Multiple threads update a shared variable concurrently to demonstrate a race condition.

Source code:

```
#include <stdio.h>
#include <pthread.h>

int counter = 0;

void *increment(void *arg)
{
    for (int i = 0; i < 100000; i++)
        counter++;

    return NULL;
}

int main()
{
    pthread_t t1, t2;

    pthread_create(&t1, NULL, increment, NULL);
    pthread_create(&t2, NULL, increment, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("Final Counter Value: %d\n", counter);

    return 0;
}
```

Output:

```
user@LAPX-0934-890:~$ nano counter.c
user@LAPX-0934-890:~$ gcc counter.c -o counter -pthread
user@LAPX-0934-890:~$ ./counter
Final Counter Value: 200000
```

Observation: The multithreaded counter program was executed using POSIX threads. Multiple threads updated the same shared counter simultaneously, resulting in a **race condition**. The final counter value may be less than the expected value due to concurrent access.

Task-2: Modify the previous program using `pthread_mutex_lock()` and `pthread_mutex_unlock()` and compare the output with the race-condition version.

Source code:

```

GNU nano 8.7.1
#include <stdio.h>
#include <pthread.h>

int counter = 0;
pthread_mutex_t lock;

void *increment(void *arg)
{
    for (int i = 0; i < 100000; i++)
    {
        pthread_mutex_lock(&lock);
        counter++;
        pthread_mutex_unlock(&lock);
    }

    return NULL;
}

int main()
{
    pthread_t t1, t2;

    pthread_mutex_init(&lock, NULL);

    pthread_create(&t1, NULL, increment, NULL);
    pthread_create(&t2, NULL, increment, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    pthread_mutex_destroy(&lock);

    printf("Final Counter Value: %d\n", counter);

    return 0;
}

```

Output:

```

user@LAPX-0934-890:~$ nano mutex_counter.c
user@LAPX-0934-890:~$ gcc mutex_counter.c -o mutex_counter -pthread
user@LAPX-0934-890:~$ ./mutex_counter
Final Counter Value: 200000

```

Observation: The counter program was modified using mutex locks with `pthread_mutex_lock()` and `pthread_mutex_unlock()`. The mutex provided synchronization between threads and prevented the race condition, giving the correct final counter value of 200000.